

ATFM: code and documentation architecture

Implementation baseline: 28 September 2026

Contents

Diagram atlas	1
1. Runtime topology	2
2. Request lifecycle and queue ownership	4
3. From events to forecasts and decisions	6
4. Code composition and extension points	9
5. Evaluation and observability	11
6. Documentation and publication architecture	13

[Download the PDF edition](#), with landscape diagram pages and clickable source links.

This guide maps the implementation at **28 September 2026**, including bounded prediction admission. It explains where state lives, how a request becomes evidence and a control decision, and which documents describe each layer. The diagrams describe current code boundaries; the older [context](#) and [container](#) views describe broader design intent.

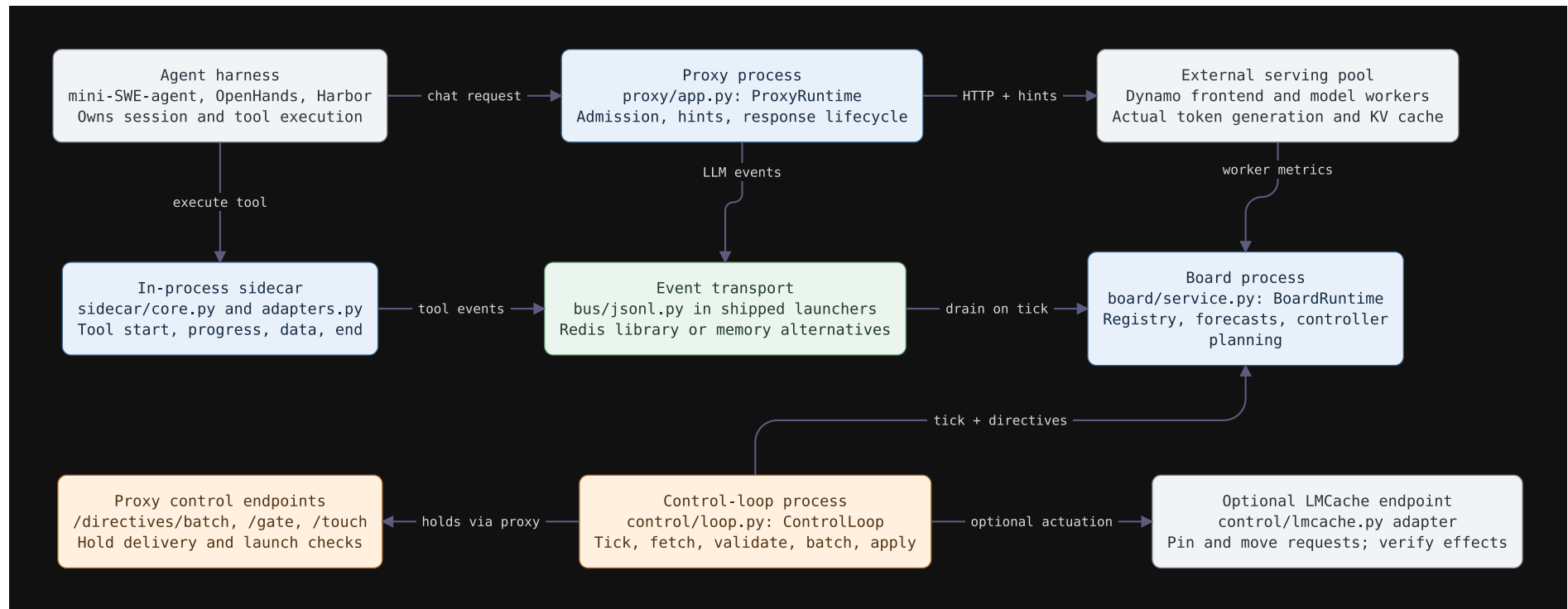
ATFM is a forecasting and admission layer around an LLM serving system. It observes agents during their tool phases, estimates when they will resume using the model, and uses that information to order or delay eligible work. The serving engine owns token generation and the actual KV tensors. ATFM owns observations, predictions, scheduling state, and control requests.

Diagram atlas

View	Question it answers	Editable source
Runtime topology	Which processes communicate, and what crosses each boundary?	reladraw
Request lifecycle	Where can a request wait, fail, or release resources?	reladraw
Forecast and control	How do tool observations turn into scheduling decisions?	reladraw
Code composition	Which modules own which responsibilities?	reladraw
Evaluation	What do replay, simulation, and HTTP trials establish?	reladraw
Documentation	Where should a change or a claim be documented?	reladraw

Open an SVG directly to zoom. Blue boxes are runtime code, orange are control responsibilities, green are data/contracts/documents, purple are evaluation/build tooling, and gray are external systems. Arrows describe the labeled interaction or dependency; they do not imply that every box is a separate process. Alternative paths and configuration-dependent behavior are called out in the text.

1. Runtime topology



Runtime topology: agent requests pass through the proxy to serving workers; sidecar and proxy events feed the board; the control loop applies decisions.

The deployed composition has three ATFM service roles: the proxy, board, and control loop. The sidecar runs inside the harness. Controller **planning** lives with the board; controller **delivery** lives in `ControlLoop`. This distinction matters when interpreting latency and failures: a board forecast, a planned hold, and an acknowledged proxy update are separate operations.

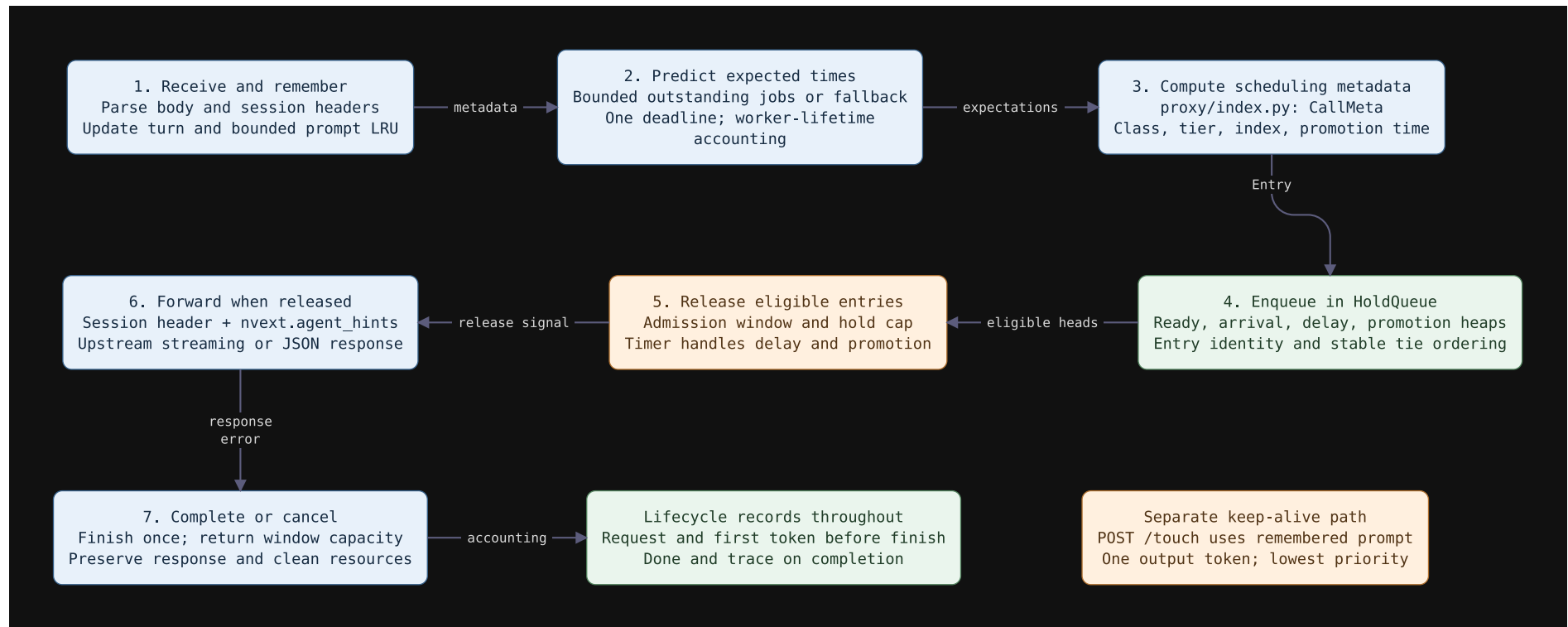
Boundary	Implementation	Responsibility and exchanged data
Harness → proxy	proxy/app.py , config.py	OpenAI-compatible chat request, session identity, class and deadline metadata.
Harness → sidecar	sidecar/adapters.py , core.py	Tool execution, result preservation, progress extraction, timeout handling. Harness-specific adapters live beside these shared modules.
Producers → bus	schema/events.py , bus/	Typed session, LLM, tool and spawn observations. The launch scripts use JSONL.
Proxy → serving frontend	proxy/app.py	Released request with <code>nvext.agent_hints</code> and session header; upstream response is streamed or returned as JSON.
Proxy → board	proxy/board_client.py	Budgeted expected service and next-tool duration used in request scoring.
Board → worker metrics	board/metrics.py , board/service.py	Configured Prometheus endpoint supplies worker capacity and cache-related signals. A frontend-only metrics page may not contain these.
Loop → board → proxy	control/loop.py , directives.py	Tick, fetch decisions, validate/expire holds, send bounded batches, validate acknowledgements.
Loop → optional cache actuator	control/lmcache.py	Pin/move requests using remembered prompt tokens. Endpoint acceptance is not proof of GPU residency or completed transfer.

HTTP surface

Routes are registered by per-application `ProxyRuntime` and `BoardRuntime` instances, avoiding shared module-level application state.

Service	Endpoint	Meaning
Proxy	POST <code>/v1/chat/completions</code>	Main admission and forwarding path.
Proxy	POST <code>/directives</code> , POST <code>/directives/batch</code>	Apply individual or batched hold updates.
Proxy	POST <code>/gate</code>	Check whether a deferrable tool/spawn should wait.
Proxy	POST <code>/touch</code>	Replay a remembered prompt with one generated token at lowest priority.
Proxy	GET <code>/session/{session_id}/prompt</code>	Retrieve remembered prompt for optional cache actuation.
Proxy	GET <code>/state</code> , GET <code>/healthz</code>	Queue state, caller outcomes, outstanding/running prediction work, and health.
Board	POST <code>/tick</code>	Drain observations and produce the next forecast snapshot.
Board	POST <code>/predict</code>	Return per-request expectations under a time budget.
Board	GET <code>/snapshot</code>	Expose forecast quantiles and metadata.
Board	POST <code>/directives</code>	Plan once for the current snapshot, then return its cached decisions.
Board	GET <code>/healthz</code>	Health response.

2. Request lifecycle and queue ownership



Request lifecycle: receive, predict, rank, enqueue, release, forward, finish and record; touches use a separate path.

ProxyRuntime parses the request, assigns session/turn metadata, remembers the prompt, and obtains expectations. Prediction failure or a missed budget falls back to local estimates. The monotonic budget includes dispatch and execution. `proxy/prediction.py` admits at most `prediction_limit` unfinished jobs (four by default); excess calls immediately use fallback. A timed-out or cancelled caller leaves its running job counted until completion. Board HTTP phases use the remaining budget. Event-loop stalls can delay timeout handling; remote computation is not forcibly cancelled.

[CallMeta and index calculation](#) translate request metadata into scheduling fields. [HoldQueue](#) owns those fields while the request waits. The application waits on an entry's release event and then forwards upstream. Completion and cancellation return admission capacity exactly once, including streaming cleanup.

The queue has several coordinated indexes rather than repeatedly sorting the entire backlog:

Structure	Purpose
Entry identity maps	Locate live entries and reject stale heap records.
Ready heap	Select highest tier, then highest index, then earliest arrival; insertion sequence resolves ties.
Arrival heap	Support arrival ordering for overflow behavior.
Delayed heap	Wake requests whose <code>not_before</code> time has arrived.
Promotion heap	Raise eligible interactive requests when their slack expires.
Directive map	Store per-session release time, reason and expiry.
Timer and release event	Connect time-based eligibility to asynchronous request handlers.

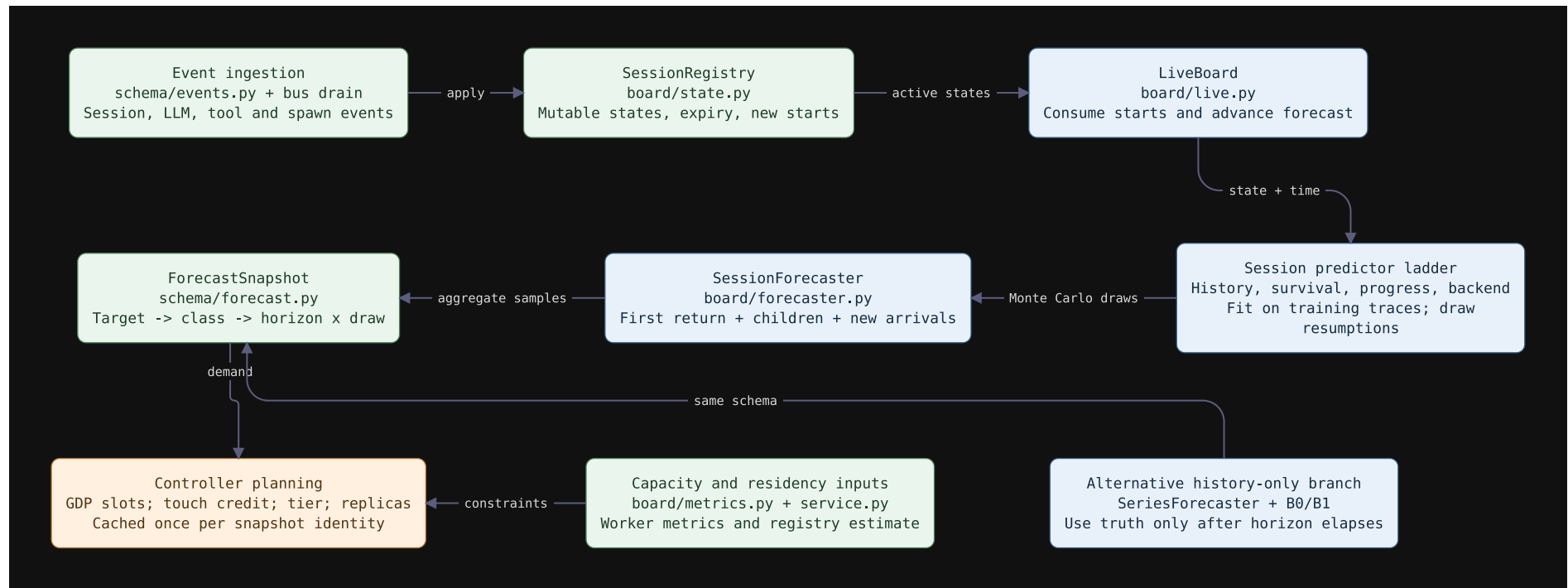
Ordinary heap insertions/removals cost $O(\log Q)$ for Q queued entries. This does **not** make every queue operation logarithmic: directive batches, diagnostic snapshots, stale-record cleanup and index rebuilds have additional work. Batched updates avoid repeating a full backlog pass for every individual hold. The queue also caps holds and has overflow behavior; it is a scheduling mechanism with explicit escape paths. The bounded prompt LRU is a record of request inputs, not a tensor cache. A touch consumes real serving work and records success only when the upstream call succeeds. Keeping it separate from the main request lifecycle makes its cost visible.

Sidecar execution boundary

The subprocess path owns process creation, streamed output, draining and process-group termination on timeout. `_ToolEvents` in [sidecar/core.py](#) shares parser dispatch with wrapped executors. Progress matches stop the parser chain; data matches can allow subsequent parsers to inspect the same line. Parser state belongs to an individual execution.

A wrapped external executor may expose output only after completion. Those observations retain completion timestamps; they cannot be interpreted as live progress samples. Only the subprocess owner may terminate its process group. Sharing parser logic does not merge these distinct lifecycle contracts.

3. From events to forecasts and decisions



Forecast architecture: event ingestion updates the registry; session predictors feed Monte Carlo aggregation; snapshots and capacity feed controllers; series models are an alternative branch.

State and transport

[SessionRegistry](#) owns current session phase, tool state, context size, activity/expiry bookkeeping and newly observed starts. [LiveBoard](#) consumes the appropriate start interval and advances the forecaster. Neither is a durable fleet database. The design assumes an owner of mutable live state; replicating the service requires an explicit state/ownership strategy.

The [JSONL reader](#) tracks file identity and byte offset. A drain reads newly appended complete records up to its observed file-size boundary. Malformed complete records are consumed and counted; incomplete trailing records wait for a later drain. Rotation and observed truncation reset the cursor. This removes repeated historical file parsing, but does not supply transactional acknowledgement or exactly-once processing.

The [Redis implementation](#) is a usable library alternative with a stream cursor and optional cursor file. It is not selected automatically by the shipped launch commands. Stream retention, cursor persistence and application of events remain separate concerns. [InMemoryBus](#) serves local composition and tests.

Prediction and aggregation

The [predictor interface](#) separates fitting, resumption draws, next-call input size and spawn prediction. Models build from history and elapsed time toward progress and shared backend factors:

- [duration.py](#): history and survival-conditioned duration models.
- [progress.py](#): progress-informed remaining work and timing.
- [backend.py](#): shared backend factors that can correlate resumptions.
- [series.py](#): history-only aggregate baselines, composed through `SeriesForecaster` instead of per-session aggregation.

For a horizon h and Monte Carlo draw m , the endogenous first-return contribution is conceptually:

$$KV(h,m) = \text{sum over sessions } i \text{ of } 1[R(i,m) \leq h] * \text{ceil}(ISL(i,m) / \text{block_size})$$

$$PF(h,m) = \text{sum over sessions } i \text{ of } 1[R(i,m) \leq h] * ISL(i,m)$$

[SessionForecaster](#) adds one level of predicted children and exogenous new-session arrivals. `Forecast-Snapshot.samples[target][class]` has shape `(number_of_horizons, number_of_draws)`. Its KV quantity is first-resumption demand, not instantaneous resident occupancy. Its prefill quantity is input-token demand, not measured cache-miss-adjusted GPU work.

The main per-session aggregation scales roughly as $O(N H M)$ for N sessions, H horizons and M draws, with additional generated-child and exogenous-arrival work. Holding H and M fixed makes that term linear in fleet size. Increasing all three simultaneously is materially more expensive. The code does not require an all-pairs session comparison for this aggregation.

[Calibration](#) and replay evaluation belong to the model/evidence pipeline. Series baselines only consume a horizon's truth after its window has elapsed, preventing future outcomes from leaking into a prediction. [Resumption quantiles](#) condition on finite draws; they do not by themselves express the probability of returning.

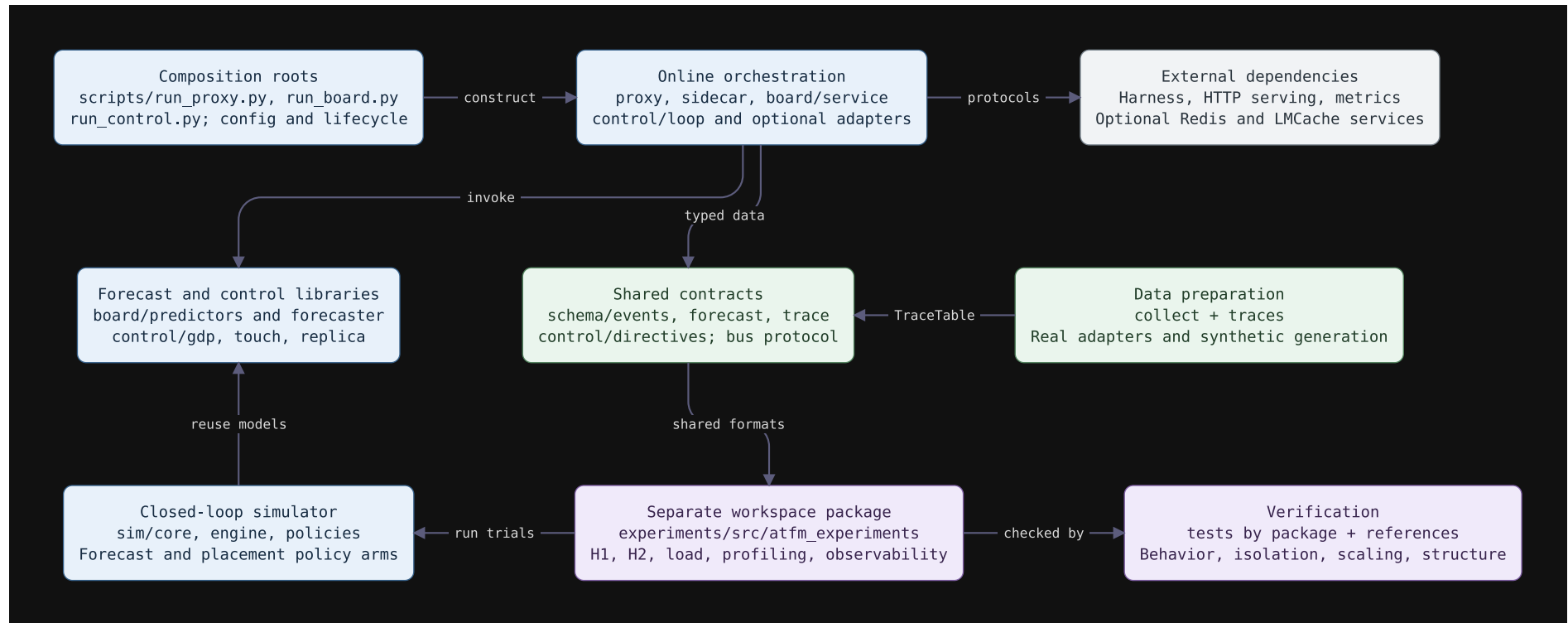
Planning and delivery

Planner	Code	Decision and limitation
GDP	gdp.py	Place deferrable sessions into slots under empirical demand/capacity constraints, delay caps and tenant accounting. Greedy allocation is a heuristic.
Touch	touch.py	Spend bounded credit on eligible cache-preservation actions using expected return and residency/frontier estimates.
Tier	touch.py	Produce placement recommendations; logged unless an actuator is configured.
Replica floor	replica.py	Recommend a minimum replica count; the launch path logs this, rather than running an external autoscaler.

GDP uses exact empirical order-statistic thresholds, indexed slot checks where applicable, and a per-plan cache of hold-window boundaries. These optimizations reduce repeated work without making joint feasibility universally logarithmic. See [control scaling](#) for precise bounds, adversarial cases and measurements. Deployment's multi-slot `GdpPlanner` and the simulator's GDP-lite policy are distinct algorithms.

Directive responses are cached by snapshot object identity because planning can consume credit or change planner state. Repeated polls must not spend that budget again. Delivery validates schemas and expiries, batches holds, and checks that the proxy acknowledgement accounts for every submitted entry. Fail-open counters are operational evidence; they are not a distributed transaction or a guarantee that a cache action completed.

4. Code composition and extension points



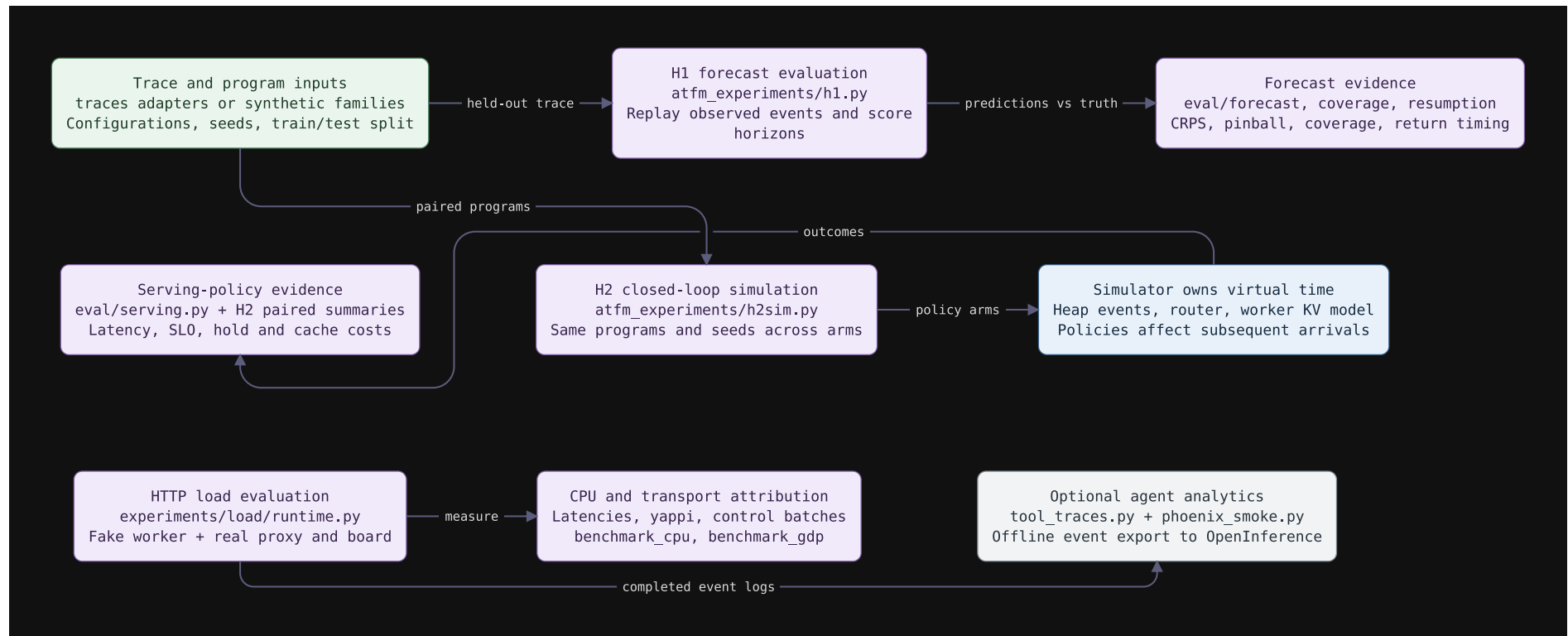
Code composition: launch scripts compose runtime modules; shared schemas connect models, traces, simulation and a separate experiments package.

Area	Owns	Change here when...
schema/	Events, canonical traces and forecast layout	A cross-module data contract changes. Keep axis and class ordering stable.
bus/	Event publication and drain semantics	Adding transport, retention or reader behavior.
sidecar/	Harness adapters, execution, gating and parsers	Supporting a harness/tool without altering tool results.
proxy/	Request metadata, queue, HTTP lifecycle, prediction client	Changing admission or forwarding. Keep resource cleanup at the owning boundary.
board/	Registry, predictors, aggregation, replay, calibration and service	Changing observed state or forecast behavior.
control/	Planning, typed decisions and delivery adapters	Adding a control action; distinguish recommendation from actuation.
collect/, traces/	Trace acquisition, conversion and synthetic families	Adding a corpus or generation recipe.
sim/	Virtual time, worker/router model and policy hooks	Studying feedback from decisions to subsequent workload.
eval/	Forecast and serving metrics	Changing how outcomes are scored, rather than how requests execute.
dynamo/	Local external-serving launch support	Changing local integration setup.
experiments/	Trial orchestration, load generation, profiling and optional analytics	Adding a study or benchmark. Core runtime must not import this workspace package.
scripts/	CLI composition and maintenance entry points	Exposing library behavior as a command.

This is a responsibility map, not a claim that every import forms a strict layer. Simulation intentionally reuses forecasting and scheduling libraries while owning separate virtual worker state. Experiments depend on `atfm`; [package-boundary tests](#) guard the reverse direction.

Functions stay within the project's 20-physical-line limit, checked by [check_function_size.py](#). Small functions should name meaningful operations and leave lifecycle ownership visible. Avoid splitting invariants across unrelated helpers merely to satisfy the line count. See the [core developer guide](#) for ordering, RNG, parser, queue and controller contracts.

5. Evaluation and observability



Evaluation paths: H1 scores held-out forecasts, H2 compares policies in a closed-loop simulator, and HTTP load trials measure runtime overhead with optional profiling and analytics.

There are three different evidence paths:

1. **H1 replay** fits on training data, advances held-out events and scores forecasts against later observations. It measures forecast quality. It does not show how a different policy would have changed the recorded trajectory.
2. **H2 simulation** runs paired programs/seeds under alternative policy arms. [Simulator](#) owns a heap of virtual events; [engine.py](#) models worker service, router affinity and cache residency. Decisions alter later request arrival times. This supports controlled policy comparisons under the model's assumptions, not GPU throughput claims.
3. **HTTP load trials** run real proxy/board/control code against a fake worker in separate processes. [load/runtime.py](#) owns setup, clients and teardown; [report.py](#) owns summaries. These trials expose CPU, serialization, queueing and transport overhead without requiring GPU hardware.

Synthetic families support large fleets without collecting an equivalently large real corpus. Iterative program generation avoids recursion depth as a fleet-tree limit while preserving seeded order. Simulator event insertion/removal is heap-based; total runtime also includes event handlers, forecasting and policy work. A cheap heap does not make an expensive tick cheap.

[CPU benchmarks](#), [GDP benchmarks](#) and [profiling](#) separate setup from the measured operations. [Performance guidance](#) explains why Python remains appropriate for orchestration and where native kernels might be justified by profiles. Moving code to Rust does not eliminate repeated scans, unnecessary HTTP calls or unfavorable scaling.

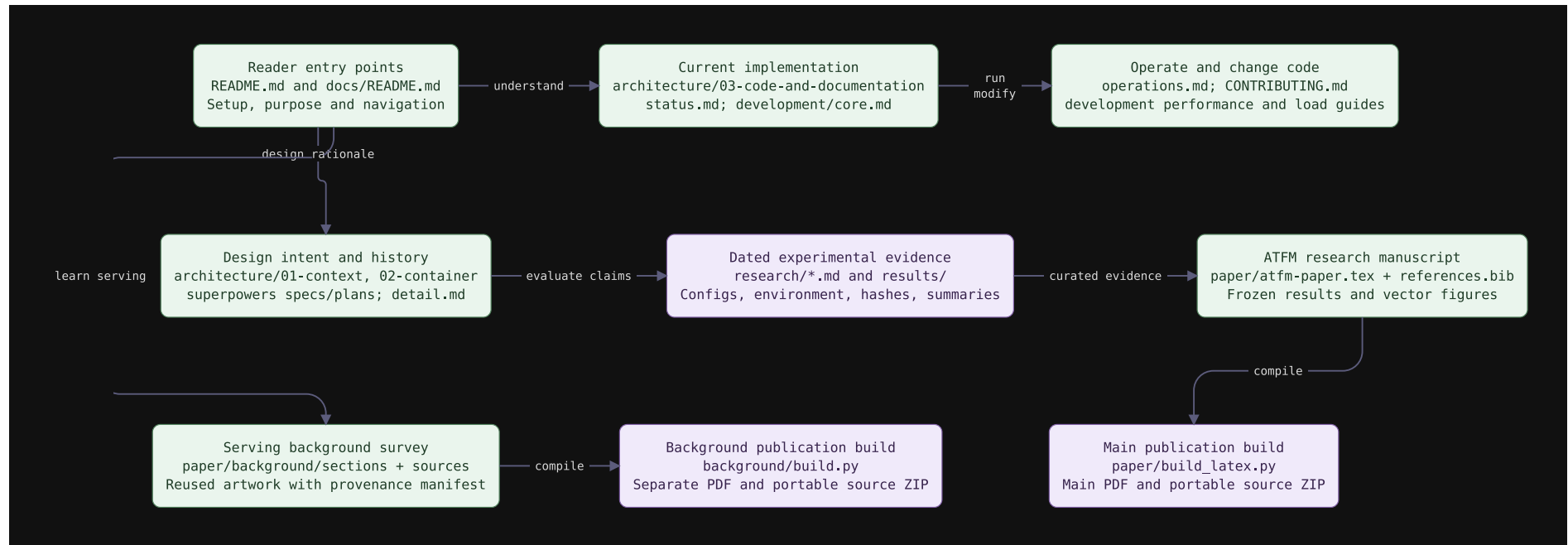
[Tool trace export](#) and the [Phoenix smoke workflow](#) are optional experiment infrastructure. They export completed event records for agent-level inspection using OpenInference/OTLP. They are not required runtime dependencies or replacements for queue/CPU measurements.

Verification map

Contract	Evidence location
Queue ordering, ties, delayed release and batch behavior	tests/proxy/ , including scan-reference comparisons
GDP equivalence, constraints and scaling cases	tests/control/ , including an original-planner reference
Event transport and malformed/partial records	tests/bus/
Parser semantics, subprocess and harness behavior	tests/sidecar/
Forecast and serving scores	tests/eval/ and board/simulation tests
Package isolation and function-size limits	test_package_boundaries.py , test_code_structure.py
Algorithmic performance contracts	tests/performance/
Measured refactor comparisons	modularity result artifacts

A passing functional test, an unchanged seeded output and a timing result establish different things. Preserve all relevant evidence when changing a hot path; do not infer GPU behavior from fake-worker trials.

6. Documentation and publication architecture



Documentation architecture: entry points lead to current implementation, operations, design records and evidence; the main manuscript and background survey have separate build paths.

Document family	Purpose	Update trigger
Root README , documentation index This guide and core development Status	First-run experience and navigation Current module boundaries and invariants Implemented versus externally validated capability	Commands, package layout or recommended reading changes. Refactor, ownership change, new endpoint or contract. New integration evidence or a discovered limitation.
Operations	Service composition and configuration	Launcher/configuration/operational behavior changes.
Performance, control scaling, load testing	Cost models and reproducible evaluation workflows	Hot-path changes or new benchmark methods.
Architecture specification, design-record index, master plan Research index and dated notes/results Main paper	Rationale, intended scope and historical decisions Workload-specific evidence with provenance ATFM hypotheses, mathematics and curated results	A deliberate design revision. Historical plans remain historical. A new experiment; retain prior results with their dates. A claim, algorithm or cited evidence changes.
Background survey	LLM serving, KV caches and related systems	Source review or tutorial updates, with a revised cutoff.

Publication pipelines

The main manuscript uses `docs/paper/atfm-paper.tex`, `references.bib`, vector figures in `latex-fig/`, and frozen evidence described by its result manifest. `build_latex.py` builds the PDF and optionally a portable source bundle. Earlier HTML material is a separate publication path and is not automatically synchronized with LaTeX edits.

The background survey has its own sections/, bibliography/source metadata, figure inputs and `build.py`. Its reproduced paper figures have a [provenance manifest](#) recording attribution and licensing. Published artwork explains those systems; the diagrams in this guide explain this repository's specific implementation.

Changing code does not regenerate either paper, and rebuilding a PDF does not validate code. A behavior change should first update implementation contracts and tests, then the relevant evidence and manuscript claims. Keep the survey's general explanation separate from local measured results.

Keeping the diagrams current

The six `.reladraw` files in [figures/](#) are the editable sources; their matching SVGs are checked-in rendered artifacts. Regenerate from the repository root:

```
for source in docs/architecture/figures/*.reladraw; do
  npx reladraw "$source" -o "${source%.reladraw}.svg"
done
```

Review the rendered output after editing. Check labels against the module/endpoint tables, distinguish process boundaries from logical modules, and keep optional or advisory paths explicitly labeled. Link a new module to its owning view rather than growing a single diagram until it is unreadable.

PDF edition

Run `docs/architecture/build_pdf.sh` from the repository root. It requires Pandoc, XeLaTeX with the standard LaTeX packages, Inkscape and DejaVu fonts. The build uses this Markdown file and the checked-in SVGs, preserves vector artwork, and writes `docs/architecture/atfm-architecture.pdf` plus a delivery copy under `output/pdf/`. `pdf-layout.lua` supplies landscape figure pages and resolves source links against the documented Git revision; `pdf-layout.tex` controls typography and page furniture. Review rendered pages after content changes.